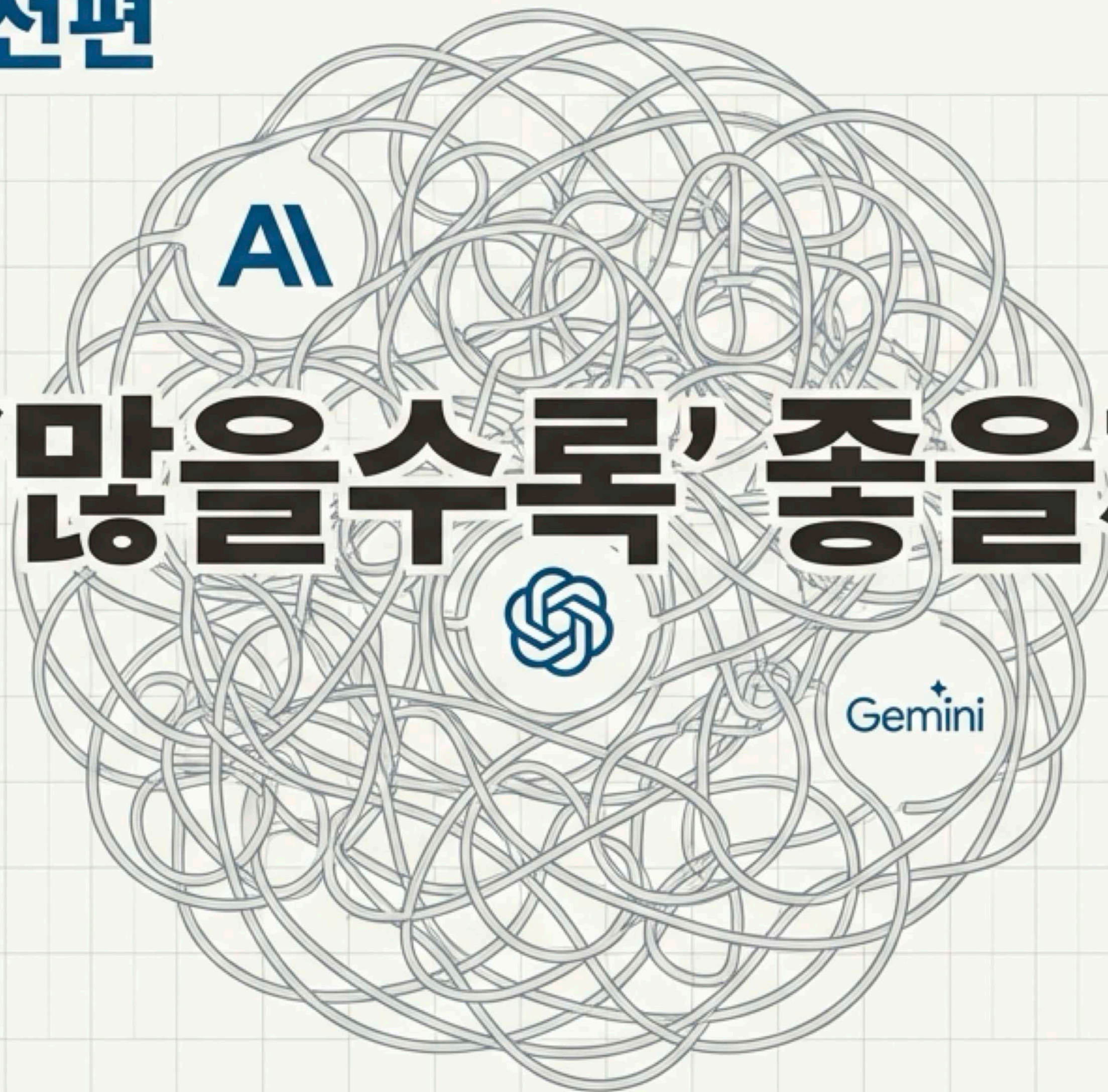


멀티 LLM을 다룰 수 있는 궁극의 워크플로우 만들어보기: 실전편

박진형, Sionic AI

정말 '마을수록' 좋을까요?





Hello, I am Sigrid Jin

<https://www.linkedin.com/in/sigridjineth/>

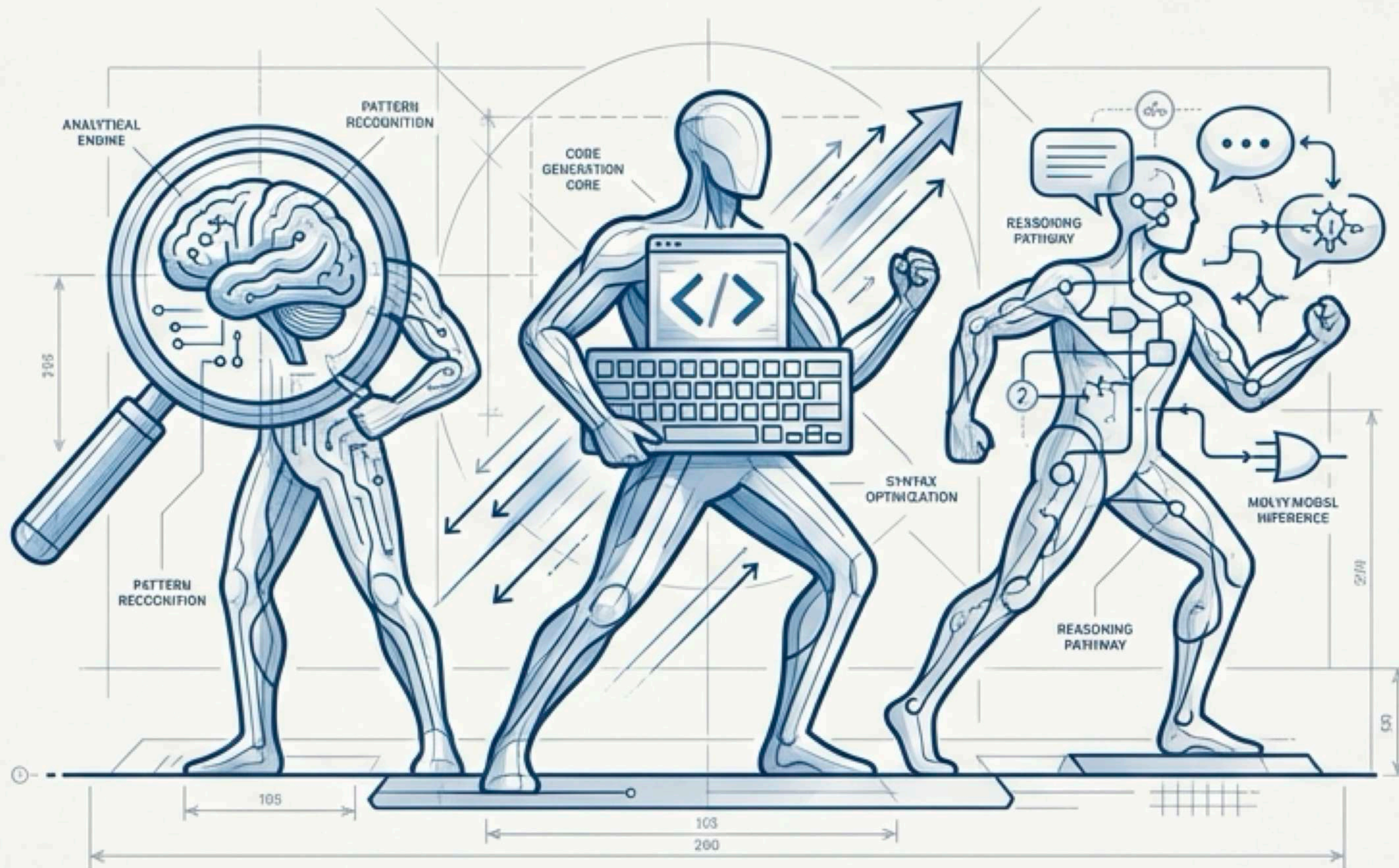
<https://www.instagram.com/sigridjineth/>

- ✗ I am not Machine Learning engineer
neither Frontend, Backend Engineer
- ✓ I am AI-powered Ultrathink Engineer
- ✓ ... which covers all span of the tech
that contributes to \$\$\$

- ✓ I am working for Sionic AI, building
enterprise-grade Agentic solution
- I have my academic base in
- ✓ University of British Columbia, but
yet to graduate

우리 모두가 꾸는 꿈: AI 어벤져스 팀

클로드, 코덱스, 제미니를 한 팀으로 묶으면,
어떤 코드든 해결하는 슈퍼 에이전트를 만들 수 있지 않을까?



개발자라면 누구나 이런 상상을 합니다.
각기 다른 강점을 가진 LLM들을 조합하면
코딩의 모든 문제를 해결할 '어벤져스'
팀이 탄생할 것이라는 기대.
분석은 **Claude**, 코딩은 **Codex**,
추론은 **Gemini**에게 맡기는
완벽한 분업 시나리오.

What is Claude Skills_

modular bundles of instructions +
resources that run in the code-
execution container

progressive disclosure: metadata first,
then exploring deeper files when
needed

skills = capabilities of your agents.
Org knowledge is one use case.

PLATFORM SKILLS

- frontend-skills
- design-figma-skills
- docx, pptx, xlsx generation

YOUR CUSTOM SKILLS

- workflow-automation-skills
- organization-internal-knowledge ★
- api-integration-patterns

Your Custom Skills: Files in Git

```
skills/ ← Git repo
├── context_editing_guide/
│   ├── SKILL.md          ← Entry point (metadata)
│   └── strategies.md     ← Pulled in on demand
├── memory_playbook/     ← Our Memory implementation
│   ├── SKILL.md          guide (NOT the feature)
│   └── fintech_patterns/ ← Team learnings
```

SKILL.md frontmatter:

```
---
name: context-editing-guide
description: Managing context window, token optimization...
---
```

✓ Version controlled ✓ PR reviewed ✓ Rollback ready

Towards a Science of Scaling Agent Systems

Yubin Kim^{1,3}, Ken Gu¹, Chanwoo Park³, Chunjong Park², Samuel Schmidgall², A. Ali Heydari¹, Yao Yan¹, Zhihan Zhang¹, Yuchen Zhuang², Mark Malhotra¹, Paul Pu Liang³, Hae Won Park³, Yuzhe Yang¹, Xuhai Xu¹, Yilun Du¹, Shwetak Patel¹, Tim Althoff¹, Daniel McDuff¹ and Xin Liu¹

¹Google Research, ²Google DeepMind, ³Massachusetts Institute of Technology

Agents, language model (LM)-based systems that are capable of reasoning, planning, and acting are becoming the dominant paradigm for real-world AI applications. Despite this widespread adoption, the principles that determine their performance remain underexplored, leaving practitioners to rely on heuristics rather than principled design choices. We address this gap by deriving quantitative *scaling principles* for agent systems. We define this scaling as the interplay between the number of agents, coordination structure, model capability, and task properties. We evaluate this across four diverse benchmarks: FINANCE-AGENT, BROWSECOMP-PLUS, PLANCRAFT, and WORKBENCH, spanning financial reasoning, web navigation, game planning, and workflow execution. Using five canonical agent architectures (Single-Agent System and four Multi-Agent Systems: Independent, Centralized, Decentralized, Hybrid), instantiated across three LLM families, we perform a controlled evaluation spanning 180 configurations, standardizing tools, prompt structures, and token budgets to isolate architectural effects from implementation confounds. We derive a predictive model using empirical coordination metrics, including efficiency, overhead, error amplification, and redundancy, that achieves cross-validated $R^2=0.513$, enabling prediction on unseen task domains by modeling task properties rather than overfitting to a specific dataset. We identify three dominant effects: (1) a *tool-coordination trade-off*: under fixed computational budgets, tool-heavy tasks suffer disproportionately from multi-agent overhead. (2) a *capability saturation*: we observe that coordination yields diminishing or negative returns ($\beta=-0.408$, $p<0.001$) once single-agent baselines exceed an empirical threshold of ~45%. (3) *topology-dependent error amplification*: independent agents amplify errors 17.2× through unchecked propagation, while centralized coordination contains this to 4.4×. Crucially, coordination benefits are task-contingent. Centralized coordination improves performance by 80.9% on parallelizable tasks like financial reasoning, while decentralized coordination excels on dynamic web navigation (+9.2% vs. +0.2%). Yet for sequential reasoning tasks, every multi-agent variant we tested degraded performance by 39–70%. The framework predicts the optimal coordination strategy for 87% of held-out configurations, providing a quantitatively predictive principle of *agentic scaling* based on measurable task properties.

SOLVING A MILLION-STEP LLM TASK WITH ZERO ERRORS

Elliot Meyerson*
Cognizant AI Lab

Giuseppe Paolo
Cognizant AI Lab

Roberto Dailey
Cognizant AI Lab

Hormoz Shahrzad
UT Austin & Cognizant AI Lab

Olivier Francon
Cognizant AI Lab

Conor F. Hayes
Cognizant AI Lab

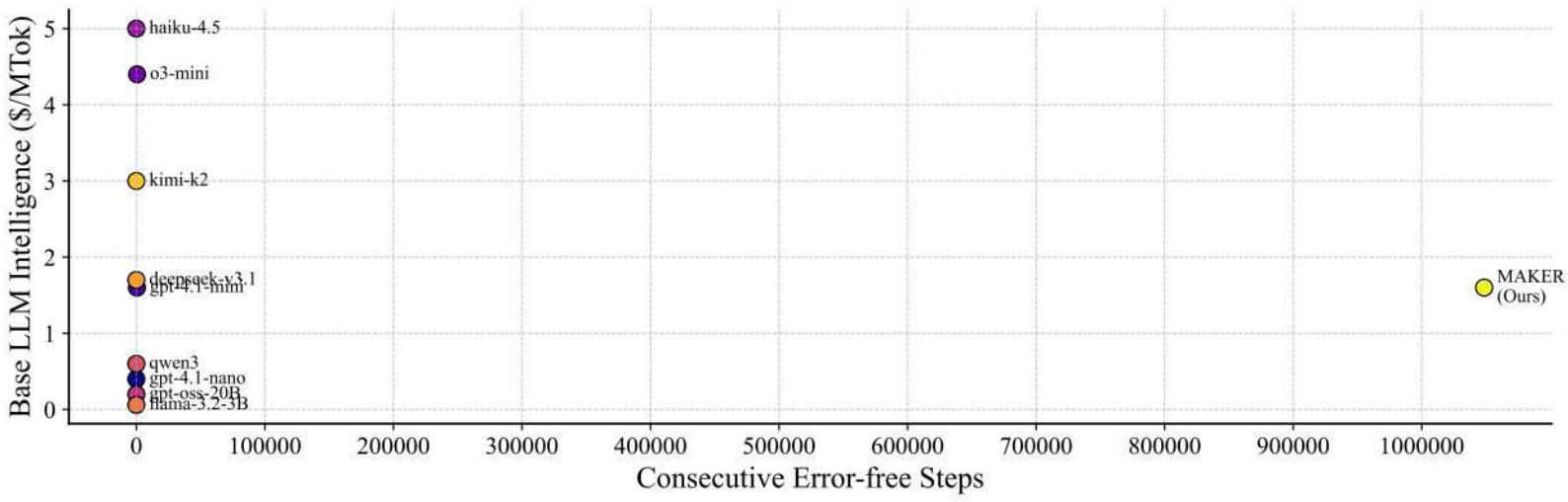
Xin Qiu
Cognizant AI Lab

Babak Hodjat
Cognizant AI Lab

Risto Miikkulainen
UT Austin & Cognizant AI Lab

ABSTRACT

LLMs have achieved remarkable breakthroughs in reasoning, insights, and tool use, but chaining these abilities into extended processes at the scale of those routinely executed by humans, organizations, and societies has remained out of reach. The models have a persistent error rate that prevents scale-up: for instance, recent experiments in the Towers of Hanoi benchmark domain showed that the process inevitably becomes derailed after at most a few hundred steps. Thus, although LLM research is often still benchmarked on tasks with relatively few dependent logical steps, there is increasing attention on the ability (or inability) of LLMs to perform long range tasks. This paper describes MAKER, the first system that successfully solves a task with over one million LLM steps with zero errors, and, in principle, scales far beyond this level. The approach relies on an extreme decomposition of a task into subtasks, each of which can be tackled by focused microagents. The high level of modularity resulting from the decomposition allows error correction to be applied at each step through an efficient multi-agent voting scheme. This combination of extreme decomposition and error correction makes scaling possible. Thus, the results suggest that instead of relying on continual improvement of current LLMs, massively decomposed agentic processes (MDAPs) may provide a way to efficiently solve problems at the level of organizations and societies.



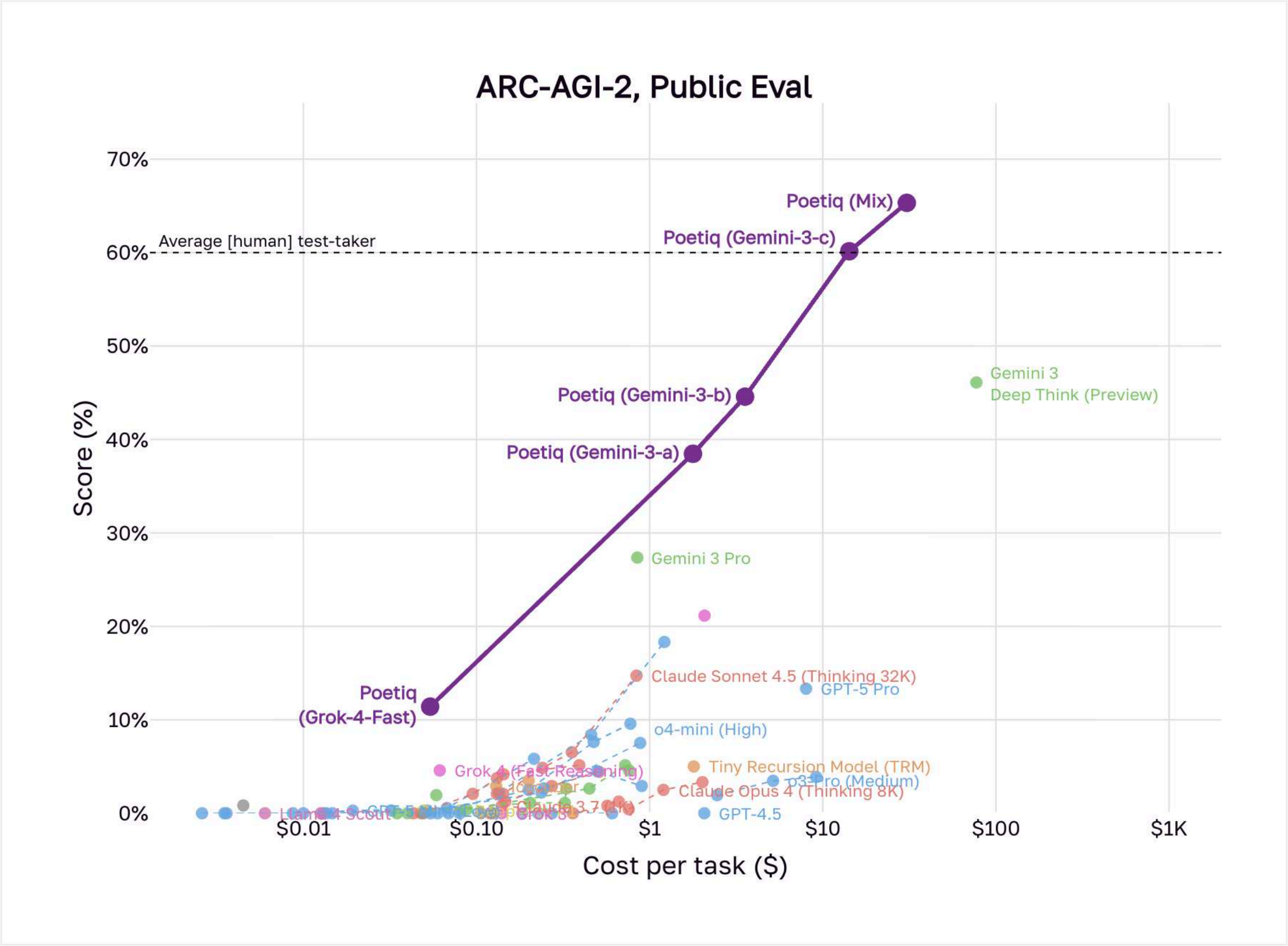


Figure 2: Poetiq’s systems redraw the Pareto frontier for cost versus performance, delivering better results for lower cost at every level on the ARC-AGI-2 Public Eval Set. ARC-AGI-2 is similar to ARC-AGI-1 but with more complex tasks.

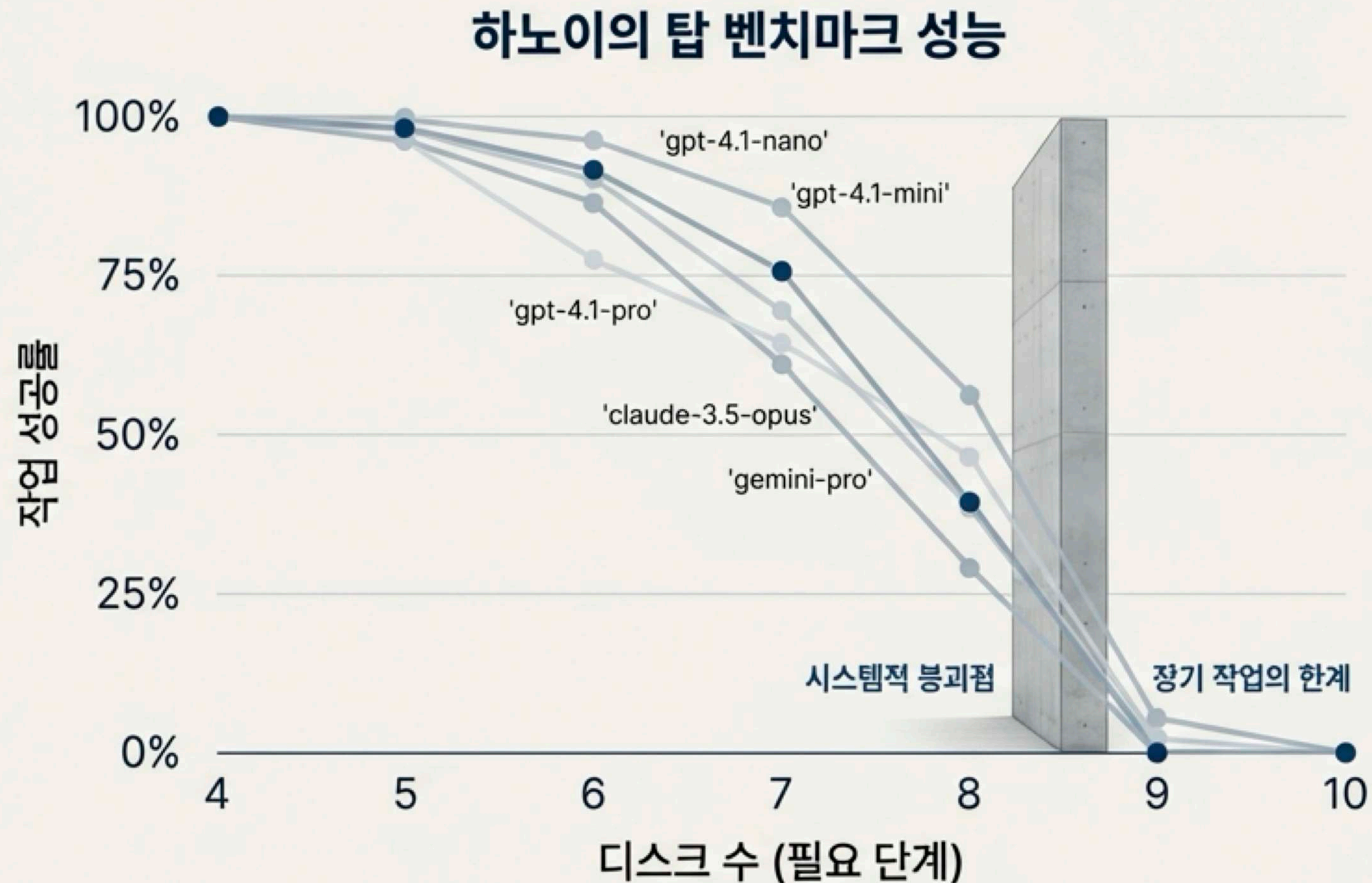
Sources: Public Eval results are from ARC-AGI official website ([public-set results](#)).

ARC-AGI-1 ARC-AGI-2

현재의 벽: 장기 작업에서 LLM은 필연적으로 실패합니다

최신 SOTA 모델도 수백 단계를 넘어가면 작업이 탈선합니다.

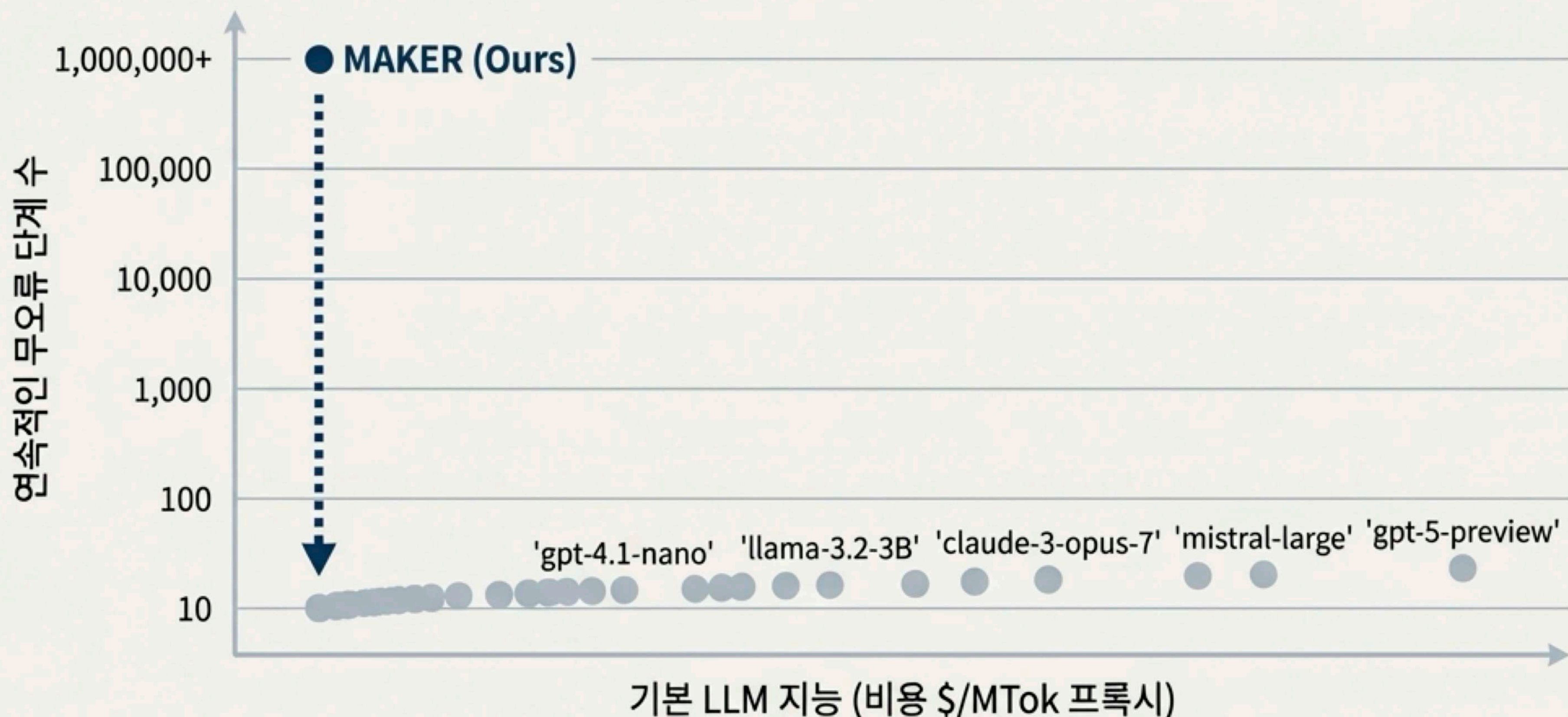
- LLM의 추론 능력은 뛰어나지만, 수많은 논리적 단계가 연결된 작업에서는 신뢰성이 급격히 저하됩니다.
- 단 1%의 단계별 오류율도 100단계 후에는 실패를 거의 보장합니다. 100만 단계 작업에서 99%의 단계별 정확도는 무의미합니다.



우리는 다른 차원의 스케일링을 간과하고 있었습니다

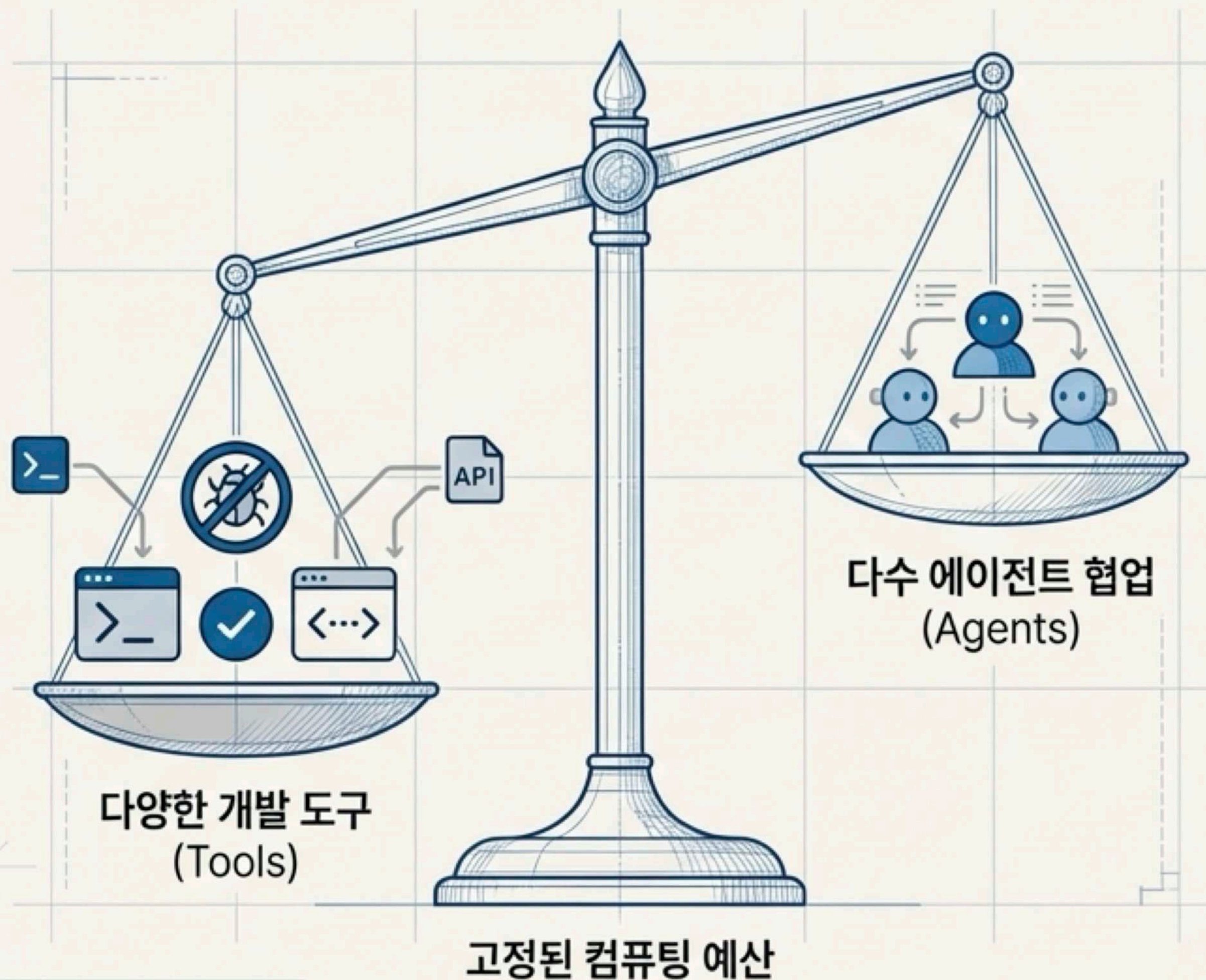
- AI 성능 확장의 지배적인 접근 방식은 더 ‘지능적인’ 기본 LLM을 만드는 것이었습니다.
- 하지만 신뢰성을 확보하는 완전히 다른, 직교하는 길이 존재합니다. 이는 모델의 지능이 아닌, **프로세스의 구조**를 바꾸는 것입니다.

AI 스케일링의 직교적 방향



법칙 1: 도구-협업 트레이드오프 (The Tool-Coordination Trade-off)

도구가 많은 복잡한 작업* 작업일수록, 멀티 에이전트 협업은 '이득'이 아닌 '비용'이 됩니다.



코딩, 디버깅, 테스트와 같은 작업은 파일 시스템, 컴파일러, 테스트 러너 등 수많은 '도구' 사용을 전제로 합니다.

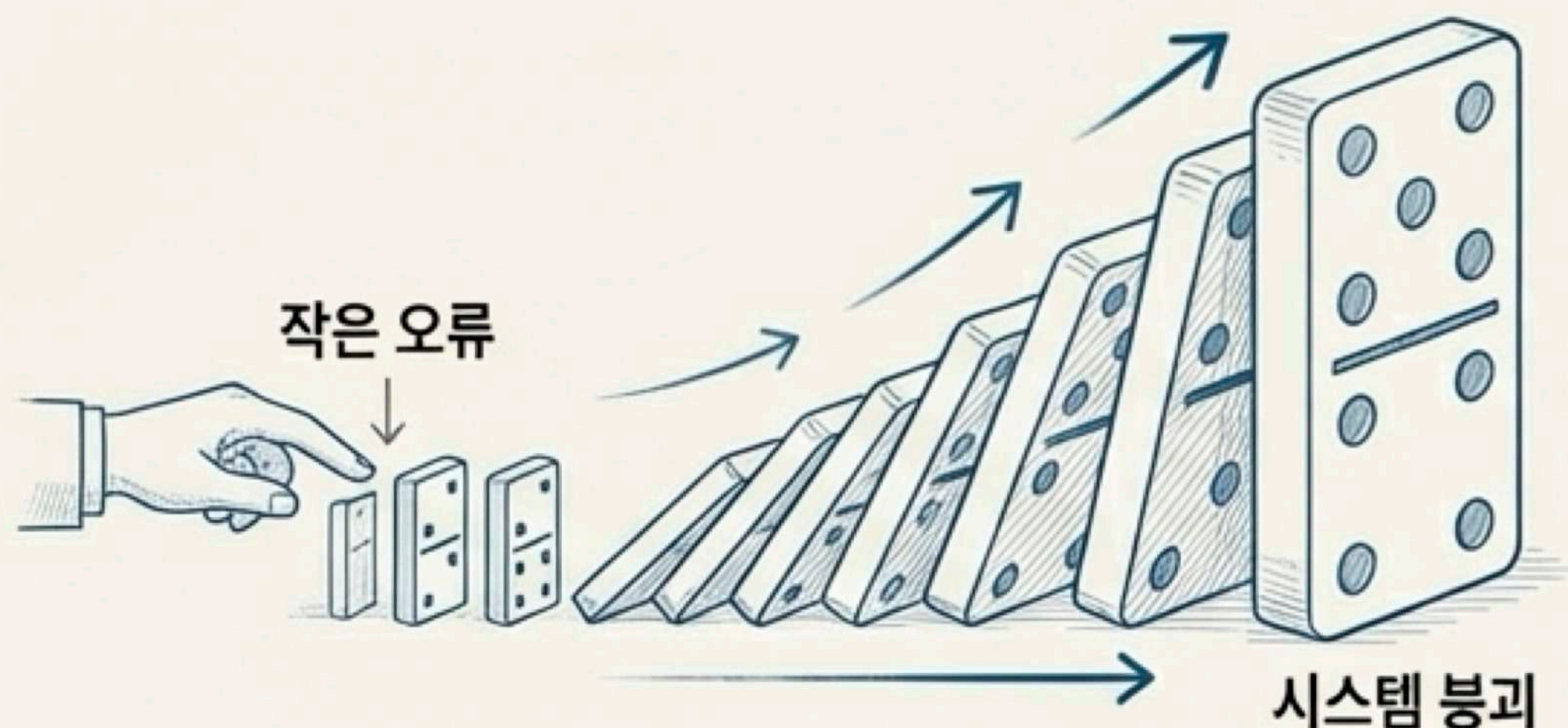
에이전트 수가 늘어나면 각 에이전트에게 할당된 **예산**(토큰, 시간)은 **줄어듭니다**.

결과적으로 **복잡한 도구 조율**(Orchestration)에 필요한 **충분한 컨텍스트**와 **추론 능력**을 확보하지 못해 **효율이 급격히 떨어집니다**.

법칙 2: 통제되지 않은 협업은 오류를 증폭시킵니다

독립적인 에이전트들은 오류를 바로잡는 대신, 서로의 실수를 증폭시켜 시스템 전체를 붕괴시킵니다.

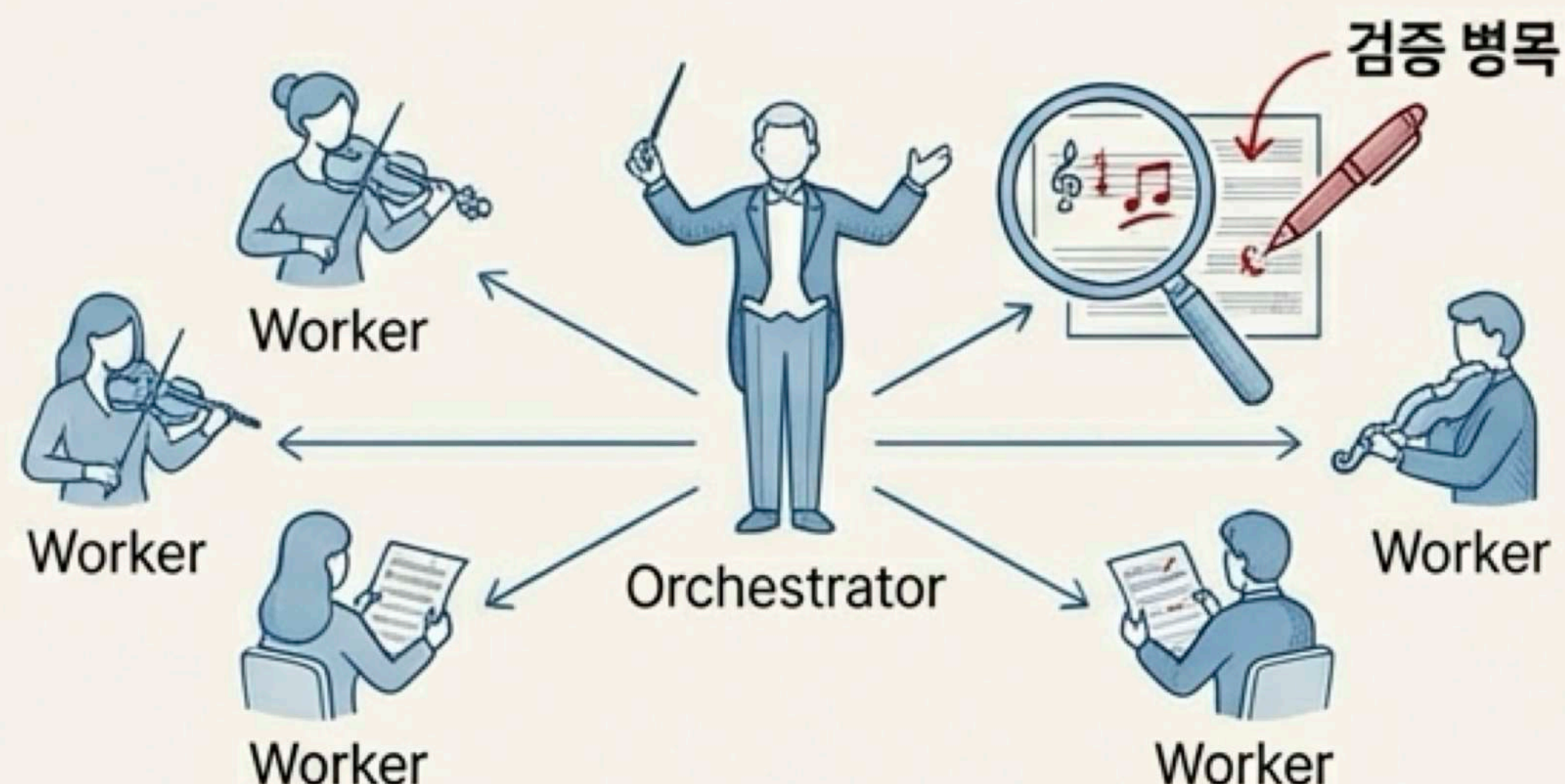
독립(Independent) 시스템



오류 증폭률: 17.2배

각 에이전트가 독립적으로 작업하고 결과를 합치면, 개별 에이전트의 작은 실수가 검증 없이 전파되어 치명적인 실패로 이어집니다.

중앙(Centralized) 시스템



오류 증폭률: 4.4배로 억제

중앙 오케스트레이터가 '검증 병목' 역할을 하여, 하위 에이전트의 결과를 통합하기 전에 오류를 잡아내고 전파를 차단합니다.

법칙 3: 유능한 에이전트에겐 ‘도움’이 오히려 ‘방해’가 됩니다

단일 에이전트의 성능이 특정 임계치(약 45%)를 넘어서면, 추가적인 협업은 성능을 떨어뜨립니다 (Capability Saturation).

이미 단일 에이전트가 문제를 거의 해결한 상태에서는, 다른 에이전트와의 소통 및 조율에 드는 비용(Coordination Cost)이 협업으로 얻는 미미한 이득보다 커집니다.

핵심: ‘지금 이 문제를 얼마나 잘 풀고 있는가?’를 측정하고, 그에 따라 협업 여부를 동적으로 결정해야 합니다.

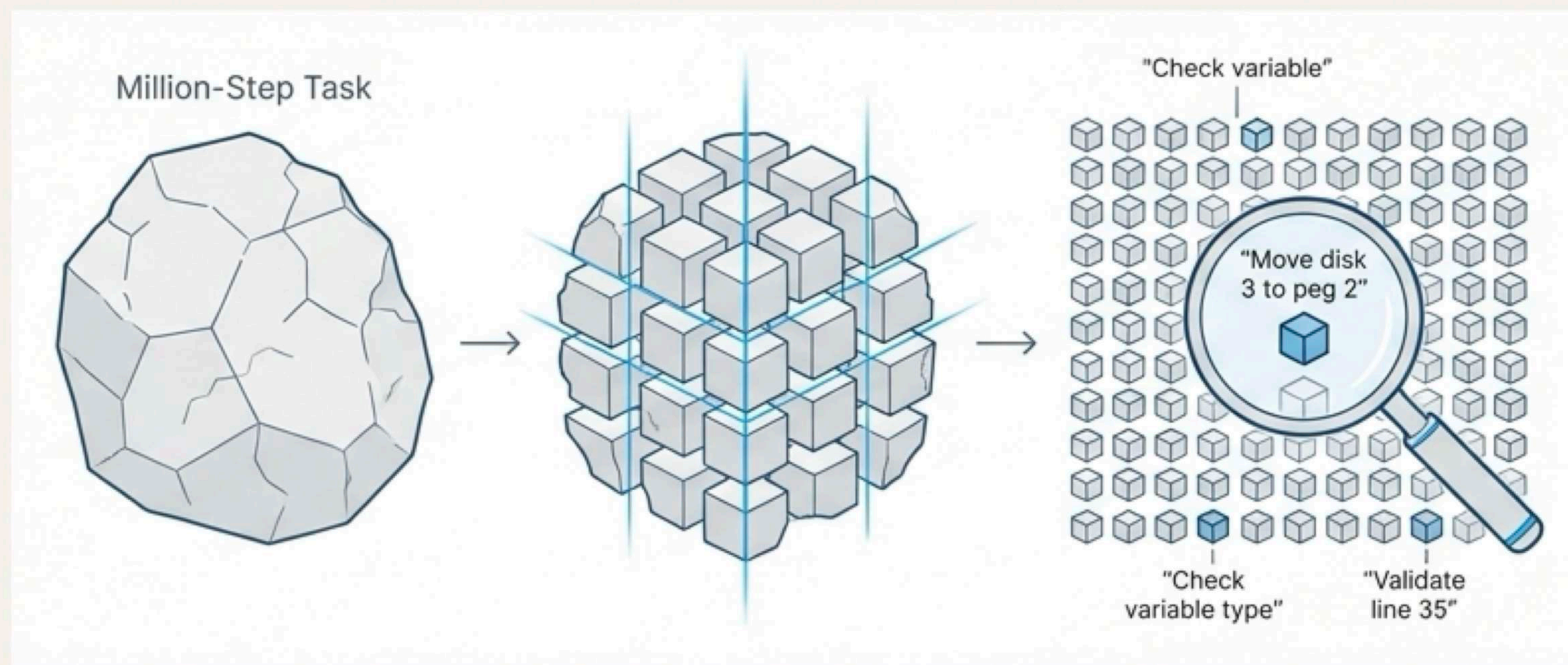


또 다른 현실: '극단적 분해'가 안정성을 만든다

오류를 피하는 가장 확실한 방법은, 일이 잘못될 수 있는 범위를 극도로 좁히는 것입니다.

'SOLVING A MILLION-STEP
LLM TASK WITH ZERO ERRORS"
연구는 새로운 패러다임을
제시합니다.

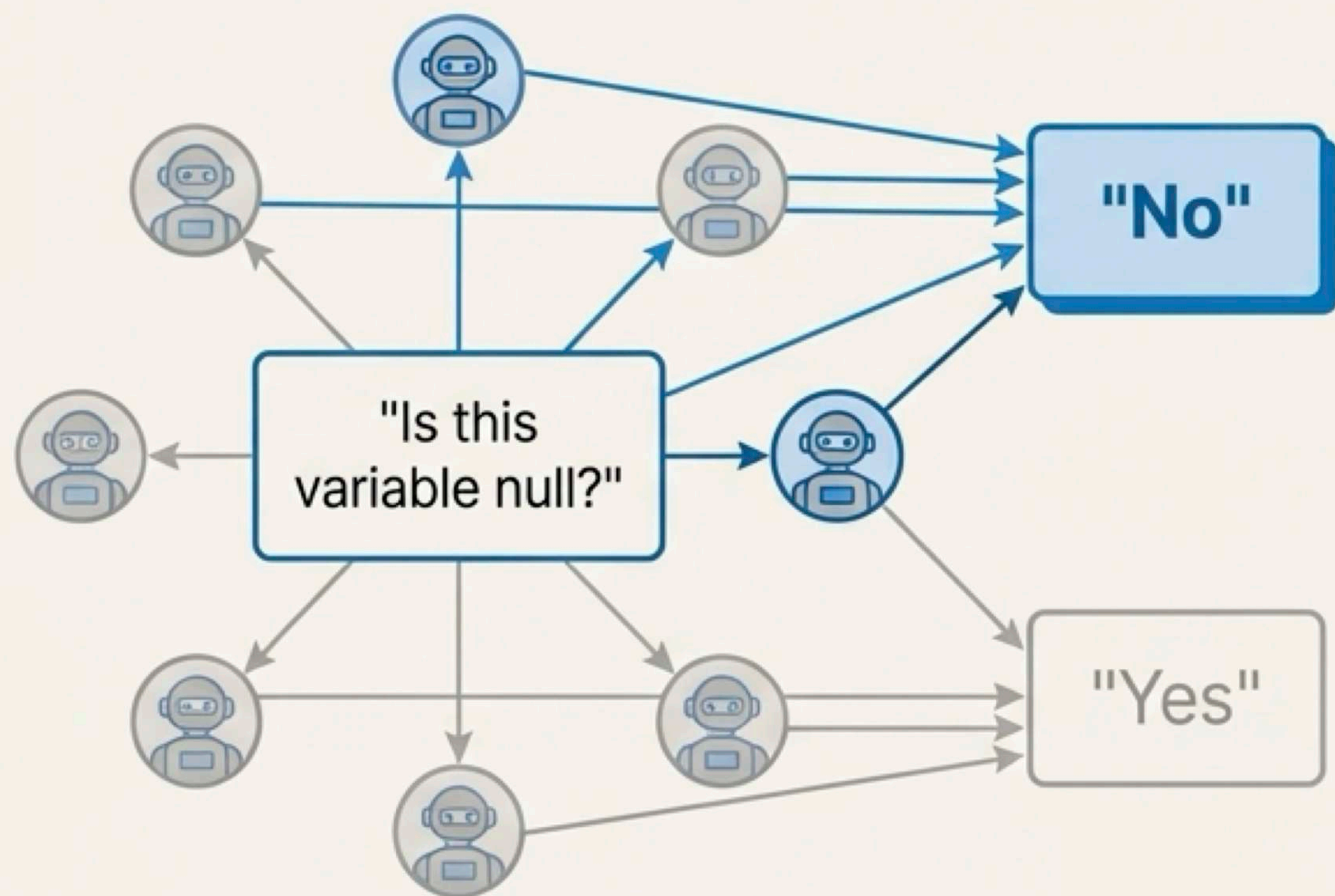
거대한 단일 작업을 수많은
'마이크로 태스크'로 분해합니다.
각 마이크로 태스크는 너무나
단순해서 LLM이 거의 실수하지
않습니다.



이 접근법은 **100만 단계의 작업을 단 하나의 오류도 없이** 성공적으로 완수했습니다.

분해된 각 단계는 '투표'로 검증됩니다

분해된 각 마이크로 태스크마다, 여러 독립적인 마이크로 에이전트가 정답에 투표합니다. 가장 먼저 K표를 앞서 얻는 답이 채택됩니다.

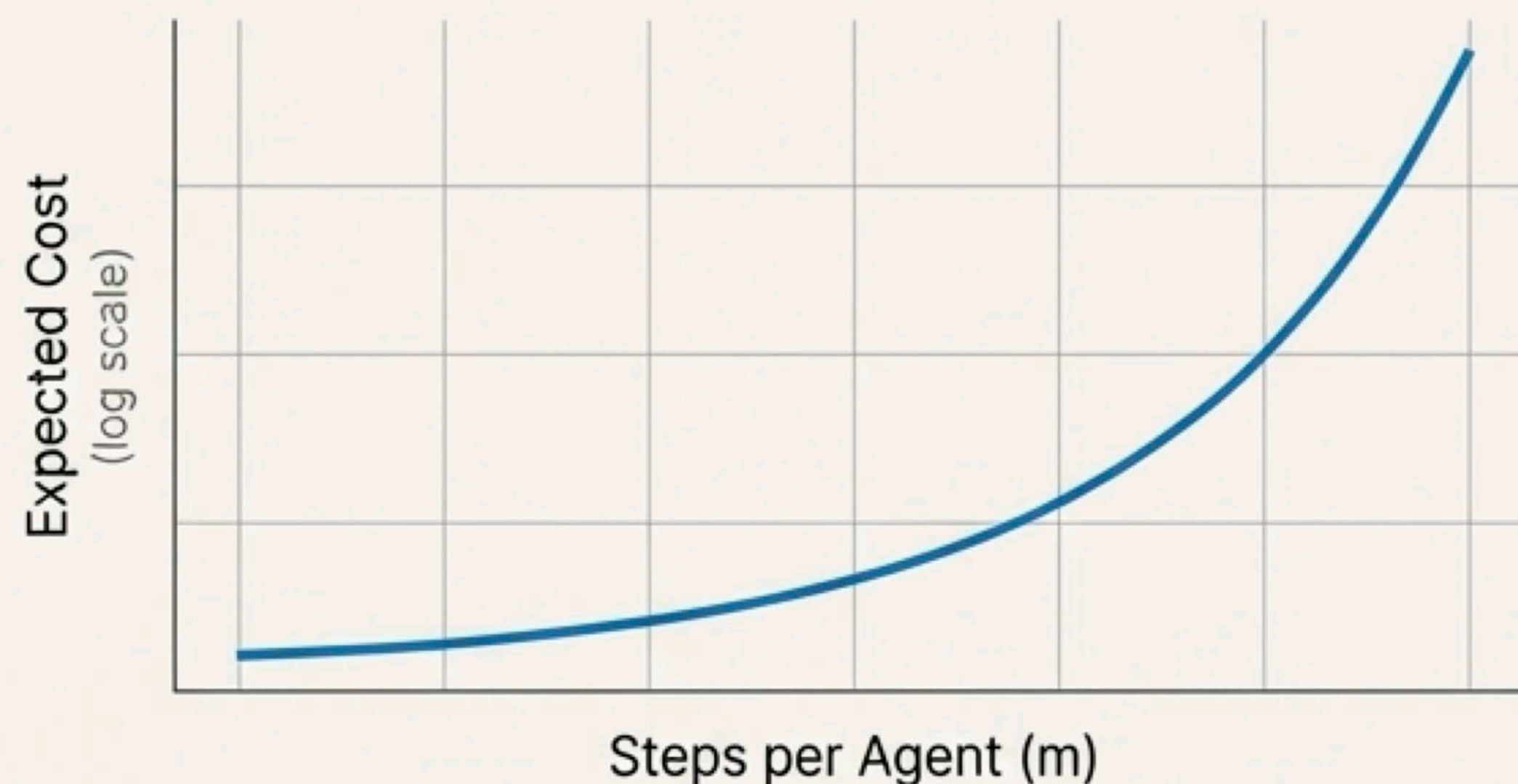


이 방식의 진짜 힘은 비용 효율성에 있습니다.

에이전트에게 맡기는 작업 단위(m)가 커질수록, 전체 작업을 안정적으로 완료하는 데 드는 비용은 **지수적으로 폭증**합니다.

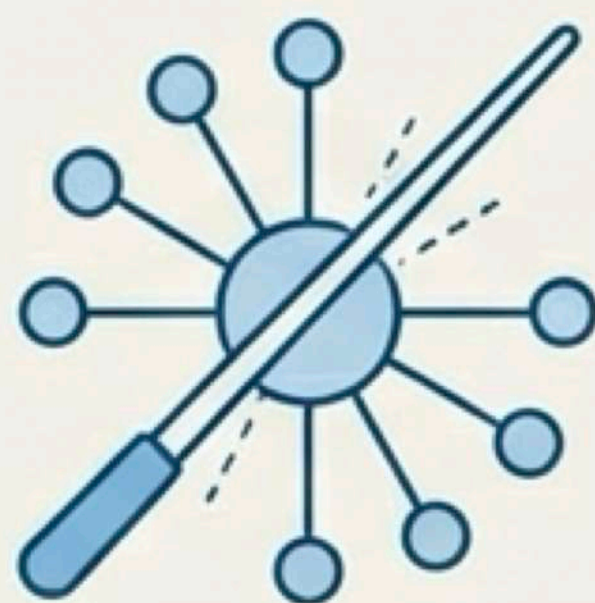
$$\text{Expected Cost} \approx \Theta(p^{-m} * c * s * \ln s)$$

결론: 작업을 가장 잘게 쪼갤 때(m=1, Maximal Decomposition), 가장 저렴하고 안정적입니다.



우리가 얻은 세 가지 핵심 원칙

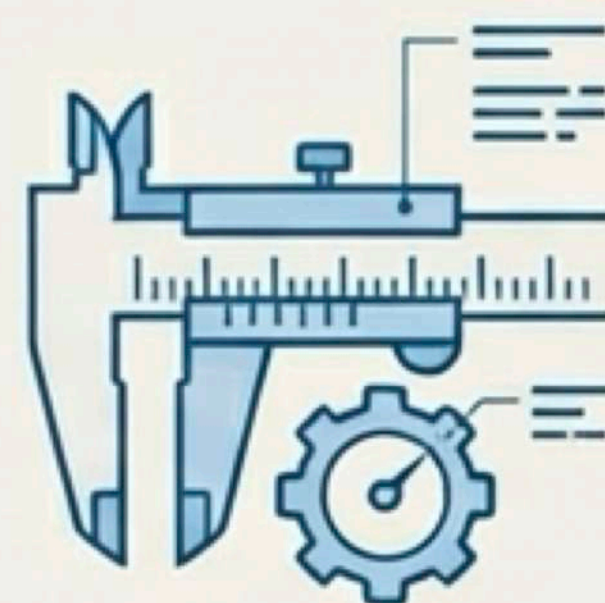
과학적 현실로부터, 우리는 멀티 LLM을 다루는 실용적인 원칙을 도출할 수 있습니다.



중앙 통제가 생명이다 (Central Control is Vital)

독립적인 에이전트는 오류를 증폭시킵니다. 항상 명확한 역할(지휘자, 실행자)을 가진 중앙 오케스트레이터가 필요합니다.

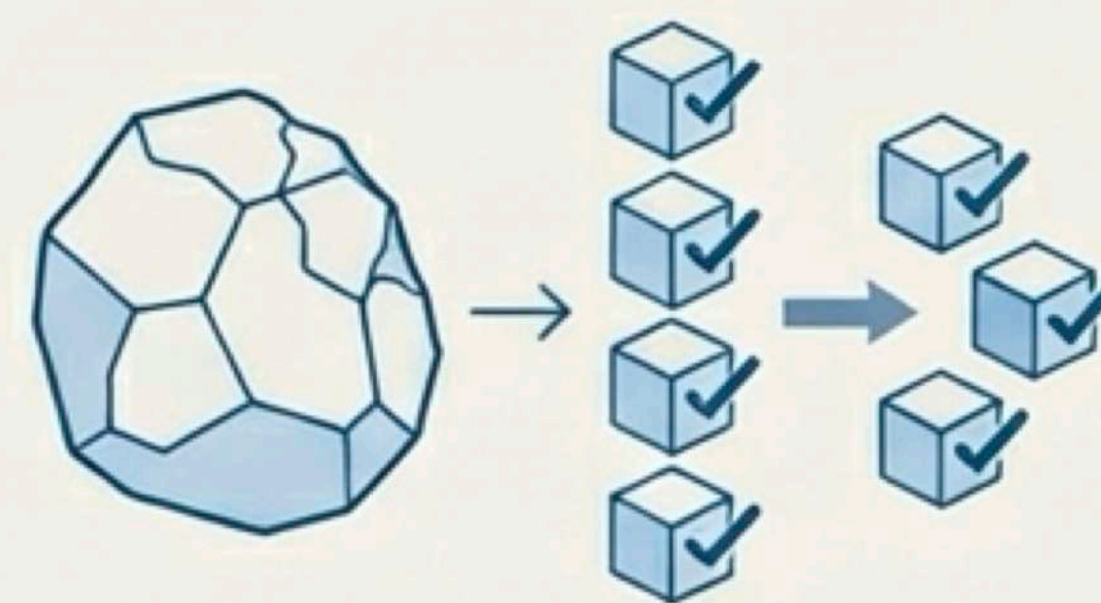
*Insight from: Scaling Agent Systems



협업은 신중하게, 측정하며 (Collaborate Deliberately & Measurably)

협업은 비싼 자원입니다. 특히 도구가 많은 작업에서는 더욱 그렇습니다. 단일 에이전트의 성능을 먼저 측정하고, 임계치를 넘으면 협업을 줄여야 합니다.

*Insight from: Scaling Agent Systems



잘게 쪼개고, 각 단계를 검증하라 (Decompose & Verify Each Step)

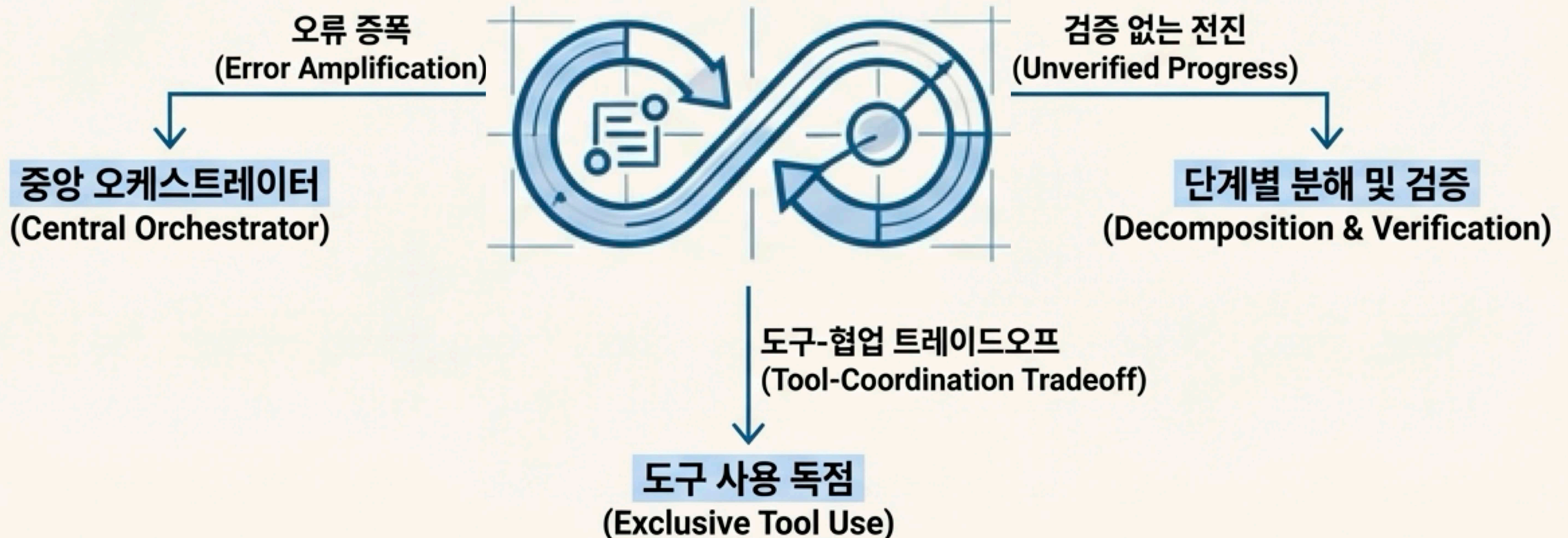
복잡한 문제를 한 번에 풀려고 하지 마십시오. 작업을 논리적인 최소 단위로 나누고, 각 단계의 성공을 확인하며 전진해야 합니다.

*Insight from: MAKER Paper

새로운 현실: 원칙에 기반한 궁극의 워크플로우, 'Zen-Loop'

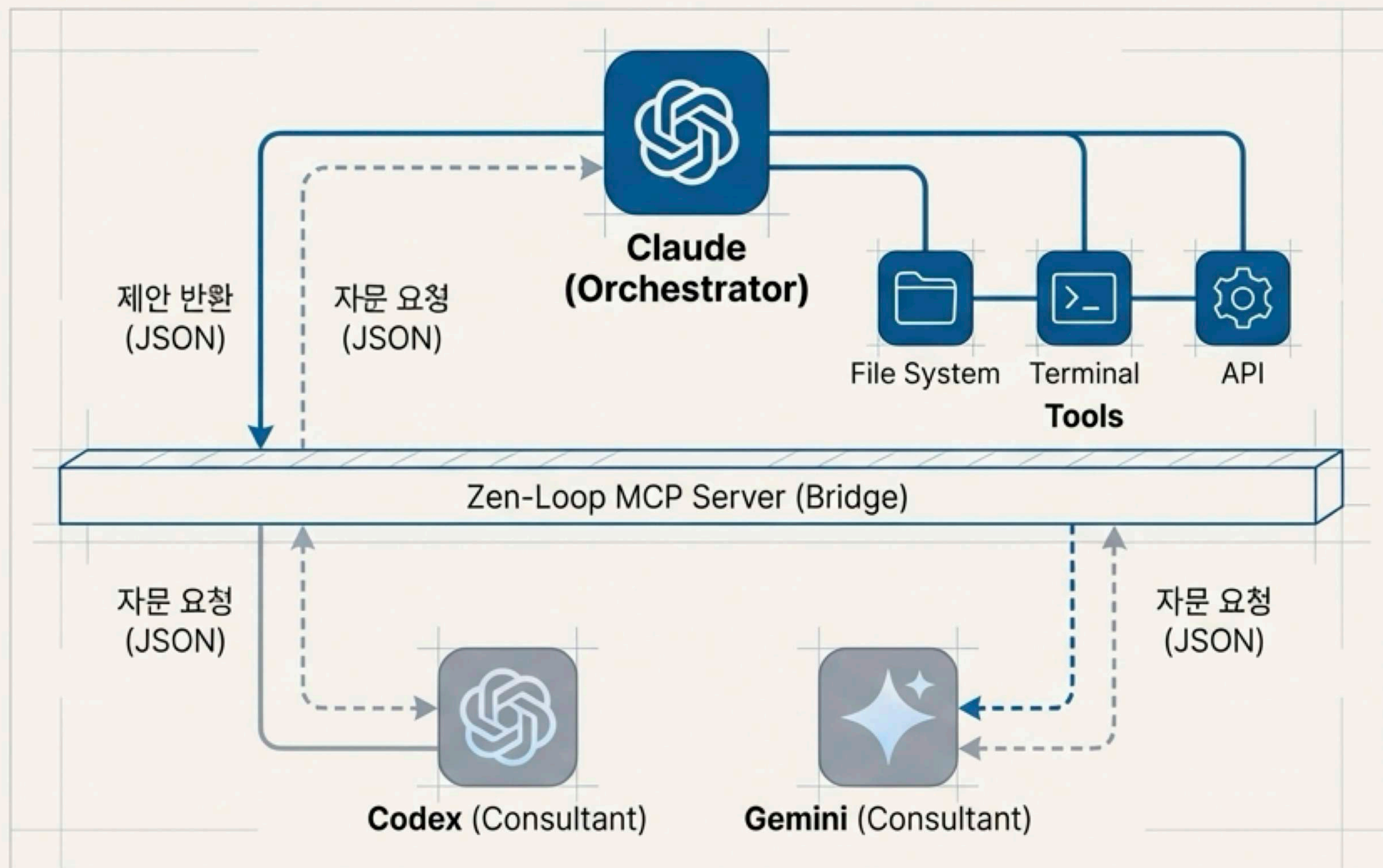
저는 이 원칙들을 코딩 작업에 맞게 구체화한 '5단계 워크플로우'와 이를 뒷받침하는 오픈소스 MCP를 제안합니다.

측정된 협업 (Measured Coordination)



철학: Claude는 '지휘자', 나머지는 '전문 자문가'

이것이 핵심입니다. Claude는 실행가이고, 다른 LLM들은 아이디어를 제공하는 자문가입니다.
도구는 오직 Claude만 사용합니다.



1. **Claude (Orchestrator)**가 문제 해결의 전 과정을 주도합니다.
2. 아이디어나 대안이 필요할 때, **MCP 서버**를 통해 다른 LLM들에게 '자문'을 요청합니다.
3. **Codex/Gemini (Consultants)**는 코드 수정, 테스트 실행 권한 없이 오직 텍스트로 된 제안(가설, 코드 스니펫 등)만 반환합니다.
4. 최종 결정, 파일 수정, 테스트 실행 등 모든 '**행동(Action)**'은 **Claude**가 전담합니다.

5단계의 'Zen-Loop' 워크플로우

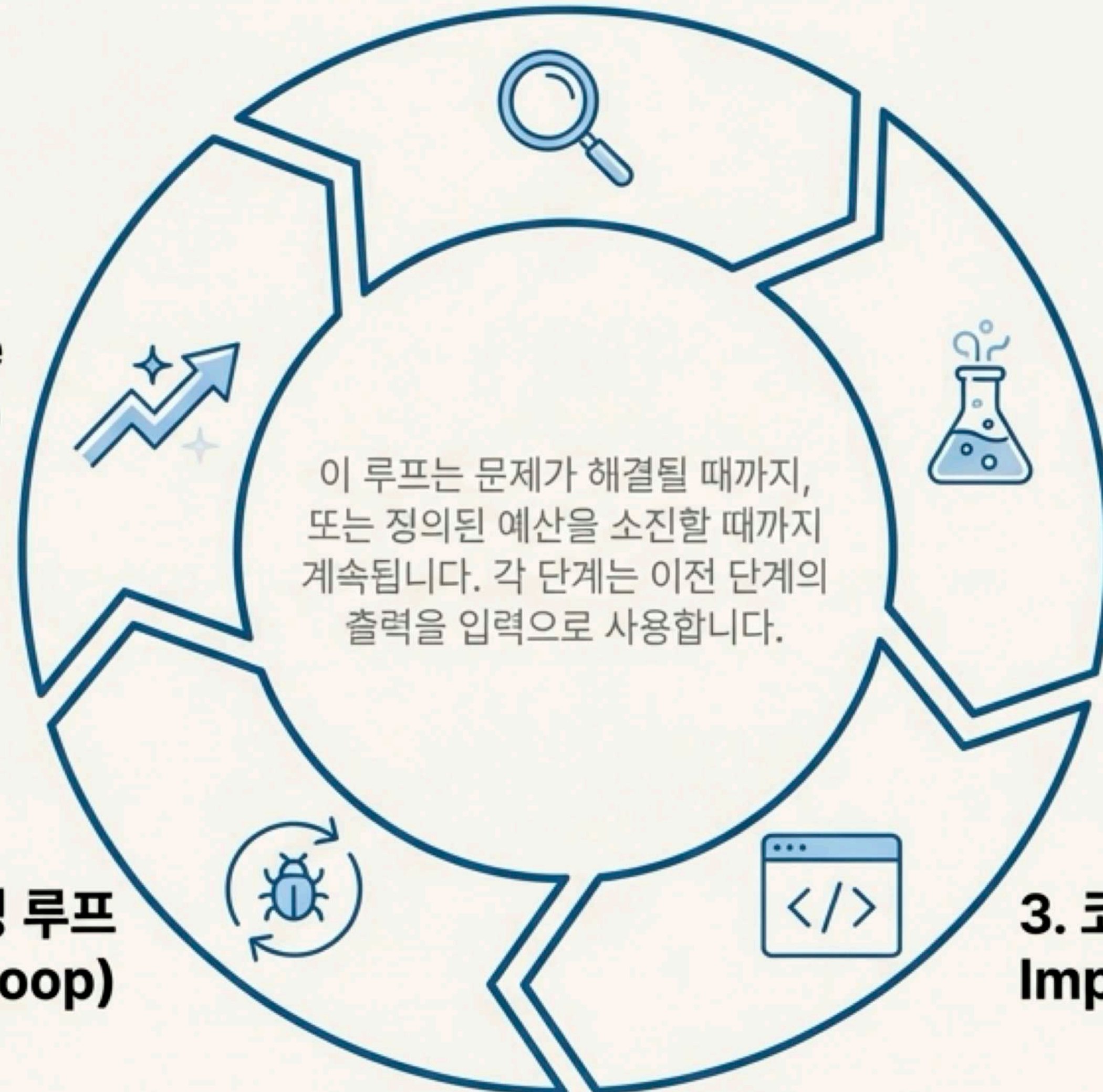
1. 예제 분석 (Example Analysis)

2. 가설 수립 (Hypothesis Formulation)

3. 코드 구현 (Code Implementation)

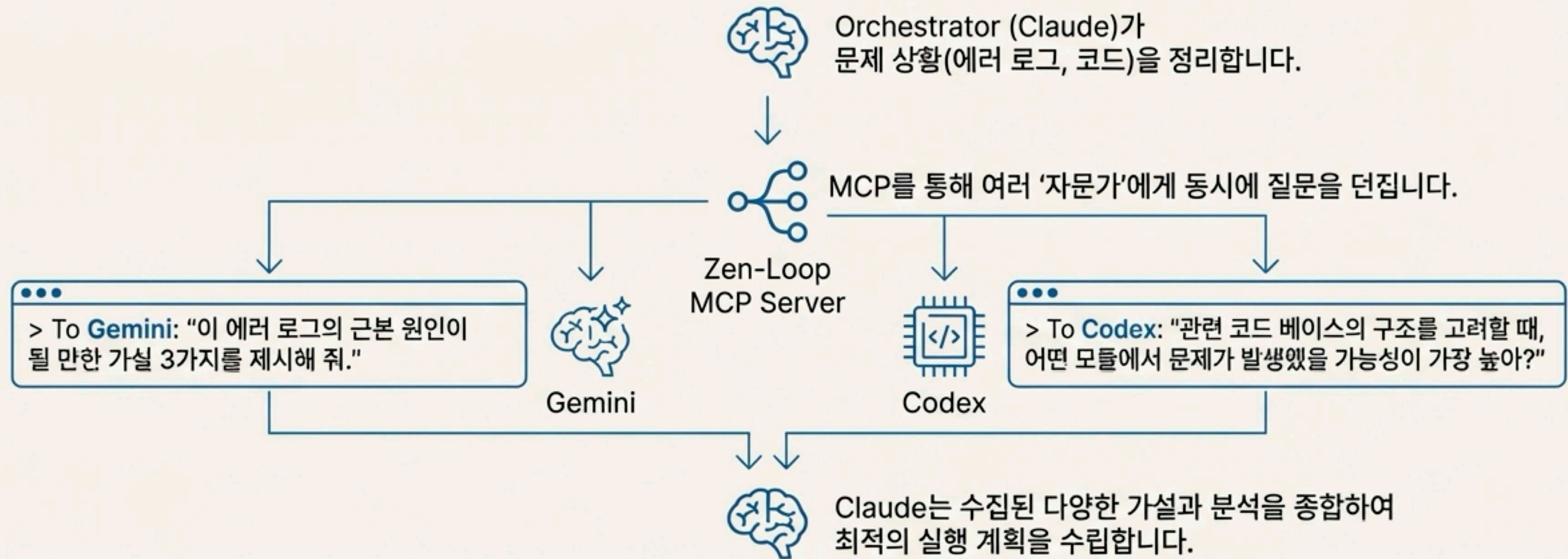
4. 반복 디버깅 루프 (Iterative Debug Loop)

5. 재귀 개선 (Recursive Improvement)



시작은 넓고 깊게: 예제 분석 & 가설 수립

단일 관점에 갇히지 않고, 다양한 가능성을 탐색하여 가장 유력한 해결 경로를 설정합니다.

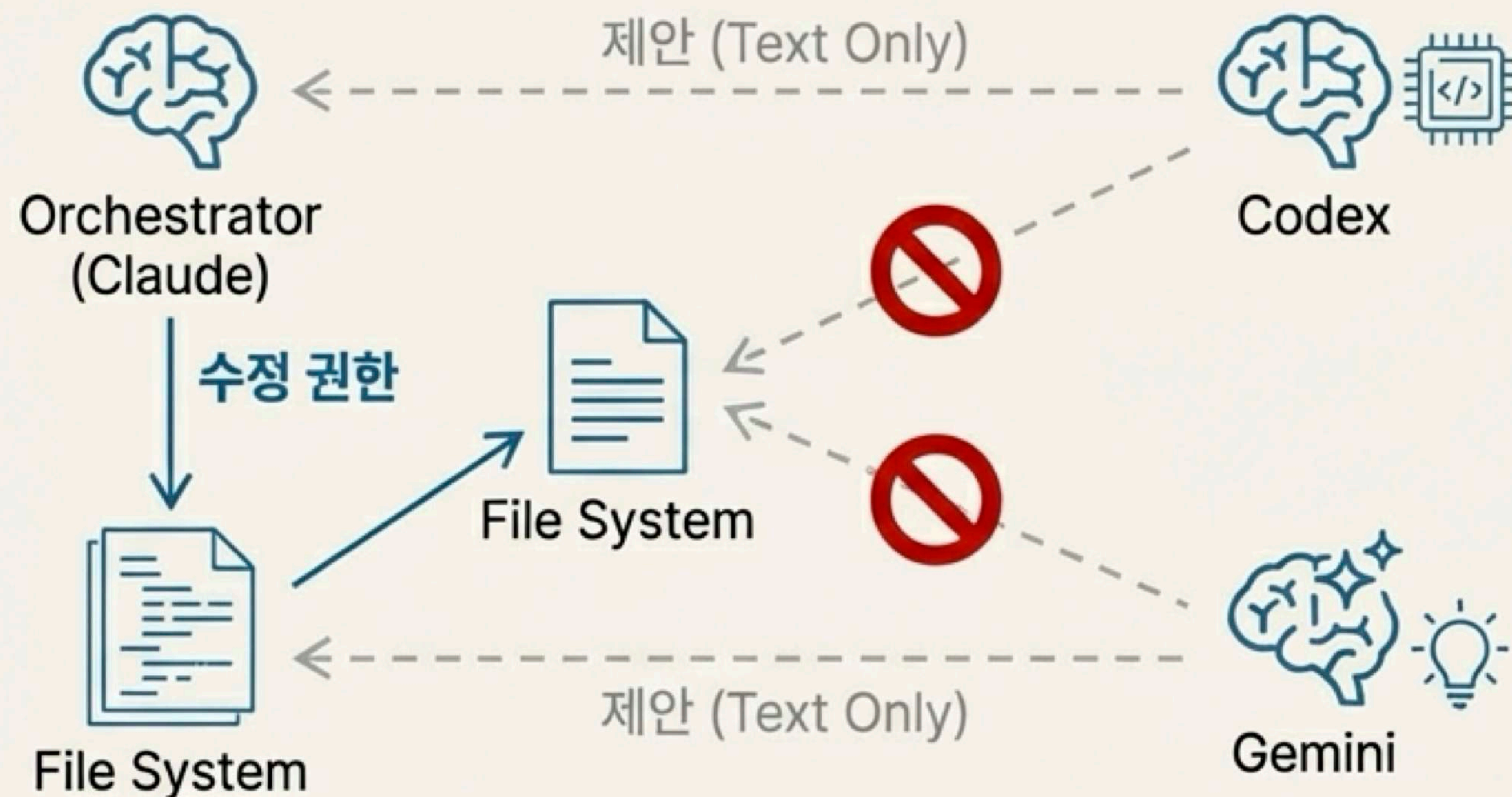


원칙 적용

- MAKER의 '앙상블/투표' 아이디어를 아이디어 생성 단계에 활용합니다.
- 도구를 사용하지 않으므로 '도구-협업 트레이드오프'를 회피합니다.
- 최종 결정은 Claude가 하므로 '중앙 통제' 원칙을 준수합니다.

실행은 단호하고 집중적으로: 코드 구현

수립된 가설에 따라 코드를 정확하게 수정하고 적용합니다.



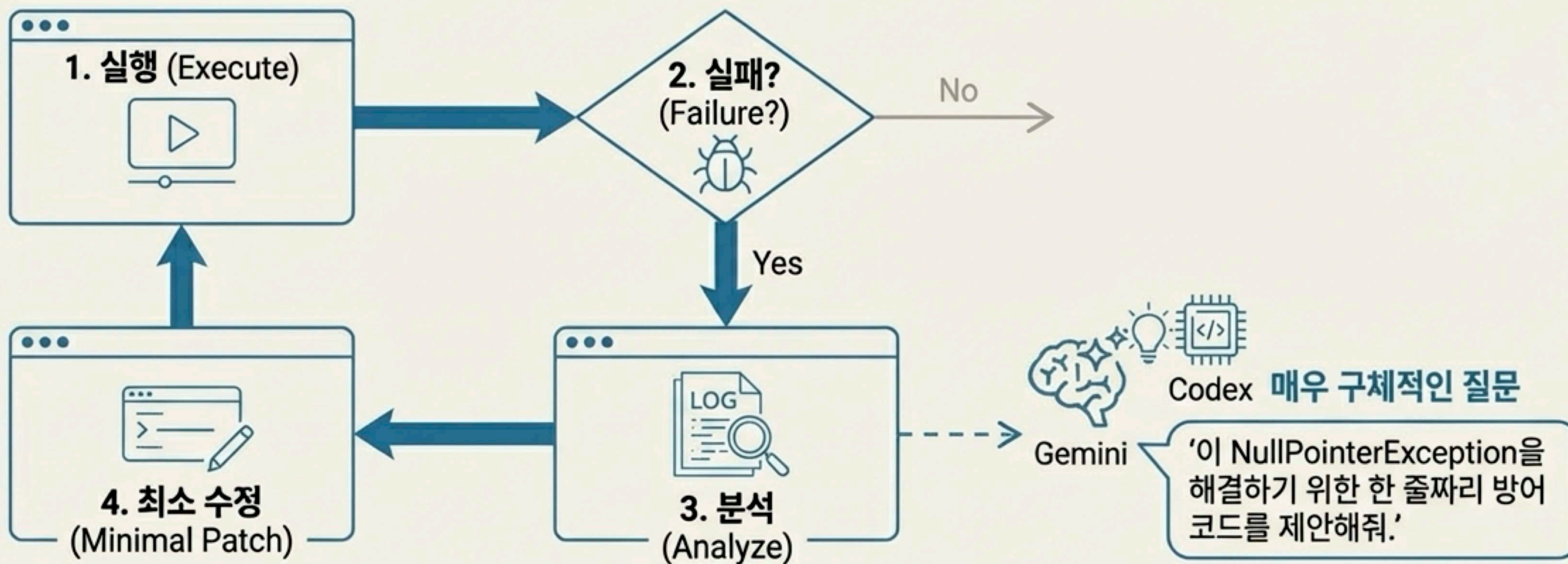
- ✓ 오직 **Orchestrator (Claude)**만이 파일 시스템에 접근하여 코드를 수정할 권한을 가집니다.
- ✓ 다른 자문가 모델(Codex, Gemini)이 코드 스니펫을 제안했더라도, Claude가 그 제안을 검토하고 직접 타이핑하거나 적용합니다.

원칙 적용

- **도구 사용 독점**: 파일 수정이라는 가장 중요한 '도구'를 단일 에이전트가 독점하여 혼란을 원천 차단합니다.
- **중앙 검증 병목 (Centralized Verification Bottleneck)**: 외부의 제안이 시스템에 직접적인 영향을 미치기 전에 Claude라는 '필터'를 거치게 함으로써 오류 증폭을 억제합니다.

가장 중요한 엔진: 반복 디버깅 루프

‘실행 → 실패 확인 → 원인 분석 → 최소 수정’의 짧은 사이클을 최대한 빠르게 반복하여 문제를 해결합니다.



원칙 적용

- **단계별 분해 및 검증:** 거대한 버그를 한 번에 잡는 대신, 테스트 실패라는 작은 단위를 반복적으로 해결하며 전진합니다.
- **순차적 작업의 중앙 통제:** 연구에 따르면 순차적 작업(A를 고쳐야 B가 보이는 디버깅)은 멀티 에이전트에 가장 취약합니다. 따라서 이 루프는 반드시 단일 지휘자가 통제해야 합니다.

끝이 아닌 새로운 시작: 재귀 개선

모든 테스트가 통과된 후, 코드의 가독성, 유지보수성, 안정성을 한 단계 끌어올립니다.



Orchestrator (Claude)가
성공적으로 수정된 코드를
최종 검토합니다.



자문가들에게 코드 품질 개선을 위한 제안을 요청할 수 있습니다.

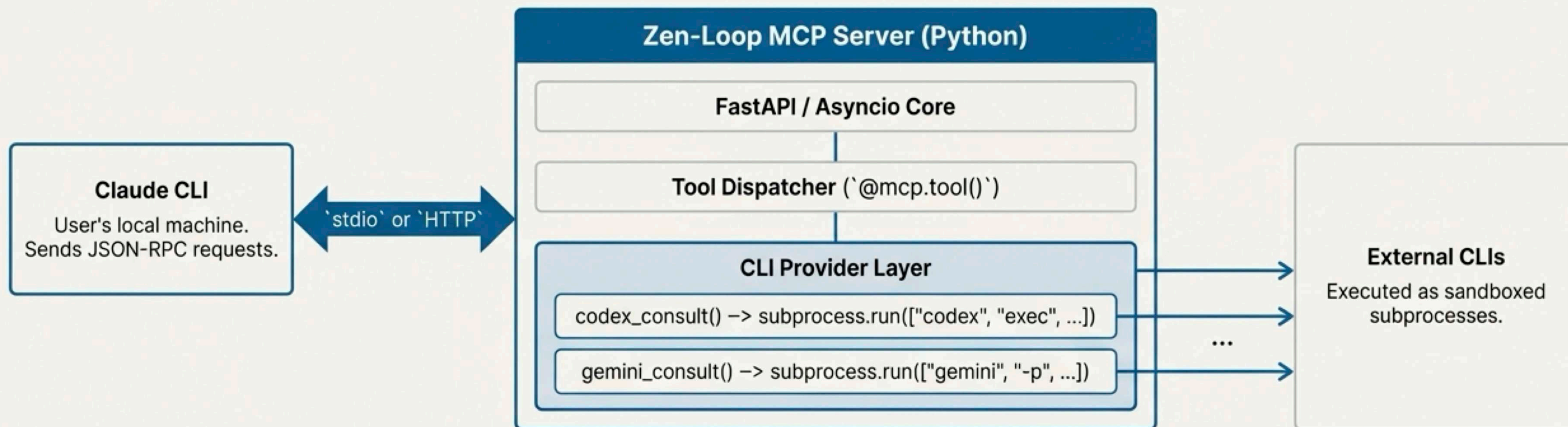
- > **To All Consultants:** "이 함수의 가독성을 높일 수 있는 리팩토링 아이디어를 제안해 줘."
- > **To All Consultants:** "이 수정사항에 대해 사용자가 알기 쉽게 주석을 추가해 줘."
- > **To All Consultants:** "미래에 발생할 수 있는 유사한 버그를 막기 위한 회귀 테스트 케이스를 제안해 줘."

원칙 적용 (Principle Applied)

핵심적인 실행(테스트 통과)이 끝난 후, 부가 가치를 높이는 안전한 영역에서 멀티 에이전트의 **창의성과 다양성**을 다시 한번 활용합니다.

내부 구조: Zen-Loop MCP 서버

복잡한 시스템이 아닙니다. Claude와 다른 CLI들 사이를 연결하는 가볍고 빠른 비동기 브릿지입니다.



Technical Details

- **프로토콜**: Model Context Protocol (MCP) 표준 준수
- **통신**: **stdio**를 통한 로컬 통신으로 빠르고 안전하게 연동
- **구현**: Python, FastMCP 라이브러리 활용
- **특징**: 상태를 가지지 않는(Stateless) 구조로 확장성과 안정성 확보

Demo

https://youtu.be/tF1wS7eV5UE

sionic-ai / sionic-maestro-skills

Public

Notifications

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

main

1 Branch

0 Tags

Go to file

Code

DolbonIn

Update README.md

3653f63 · 3 hours ago

22 Commits

.claude	feat: add explicit invocation methods for Maestro Skills	20 hours ago
maestro-mcp	Update README.md	3 hours ago
.gitignore	feat(hitl): add Human-in-the-Loop approval system for w...	20 hours ago
CLAUDE.md	feat: add explicit invocation methods for Maestro Skills	20 hours ago
README.md	fix: use correct global config path ~/.claude.json	19 hours ago

README

Maestro Skills

A Claude Code Skill implementing **Centralized Consult** architecture for Multi-LLM coding workflows. Like a maestro conducting an orchestra, Claude Code orchestrates multiple LLM models to produce harmonious, accurate output.

Based on:

- "Towards a Science of Scaling Agent Systems" (Kim et al., 2025)
- "Solving a Million-Step LLM Task With Zero Errors" (MAKER)

Key Features

- Multi-LLM Orchestration:** Coordinate Codex, Gemini, and Claude CLIs

About

Claude Code Skills implementing Centralized Consult architecture for multi-coding CLI harness.

Readme

Activity

Custom properties

5 stars

0 watching

0 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors

2

sigridjineth

Sigrid Jin (🇰🇷-🇰🇷)👤 oO

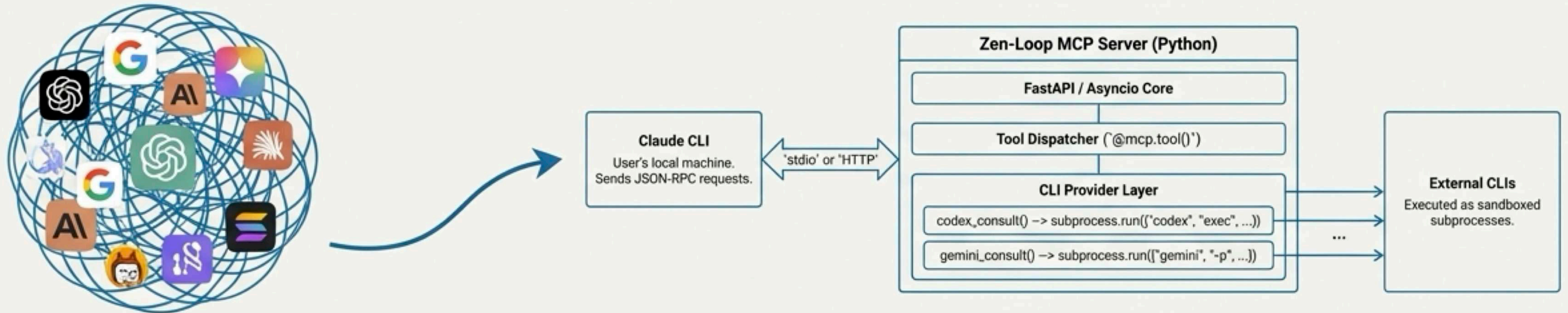
DolbonIn

Semin Jeong

Languages

궁극의 워크플로우는 '측정된 협업'을 설계하는 것입니다

더 많은 에이전트를 추가하는 것이 정답이 아닙니다. 과학적 원칙에 따라, 각자의 역할을 명확히 하고, 과정들을 측정하며, 협력의 비용과 이득을 조율하는 것이 핵심입니다.



Before: Chaos

After: Measured Coordination

함께 궁극의 워크플로우를 만들어갑시다.

<https://github.com/sionic-ai/sionic-maestro-skills>